

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Demographics:

We would appreciate your assistance in providing some basic demographic details. This information will help us understand participants' diverse perspectives and ensure that our analysis takes into account various demographic factors.

3. What is your gender?

- ☐ Man
- ☐ Woman
- ☐ Non-binary
- ☐ Prefer not to say

* 4. What is your current role?

- ☐ Software developer
- ☐ Documentation engineer
- ☐ Researcher
- ☐ Student
- ☐ Other (please specify)

* 5. How many years of professional experience do you have in software engineering?

- ☐ 0-1
- ☐ 2-4
- ☐ 5-9
- ☐ 10+

6. How experienced are you in structuring or maintaining multi-module Python projects?

- ☐ Beginner
- ☐ Intermediate
- ☐ Advanced
- ☐ Expert

*** 7. How experienced are you with producing API documentation?**

- ☐ None
- ☐ Some
- ☐ Regular
- ☐ Expert

*** 8. How familiar are you with Sphinx documentation generator tool?**

- ☐ Not familiar
- ☐ Slightly familiar
- ☐ Moderately familiar
- ☐ Very familiar
- ☐ Expert

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Sphinx (autodoc & AutoAPI):

Sphinx is the de facto Python doc generator. ``autodoc`` imports modules at build time to render docstrings (shows how names are exposed at runtime). ``AutoAPI`` (sphinx-autoapi) parses source files statically to create file-of-definition pages (shows where code lives). Both are used in the demo site you will see.

*** 9. Would you like to see an outline of Sphinx's essential components: what autodoc, autoapi, and viewcode do, and relevant config knobs (imported-members, noindex, autoapi_ignore, autoapi-skip-member)?**

☐ Yes

☐ No

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Sphinx: Traceability Components

Sphinx can be used as a traceability tool between source code and API reference docs, but the linking behavior depends strongly on the extensions and configuration you use:

- **autodoc**: imports modules at build time and extracts docstrings; it reflects runtime exposure (what users see when they import the package). Use when you want public API pages that match import-time names.
- **sphinx-apidoc**: a tool that generates automodule stubs for entire package trees. It still relies on the autodoc extension for rendering.
- **AutoAPI** (**sphinx-autoapi**): static parser that reads source files and generates file-of-definition pages without importing the package; it's safer for heavy or side-effectful projects and surfaces where code is implemented on disk.
- **viewcode**: a Sphinx extension designed to enhance the documentation of Python projects by automatically adding a bidirectional link to the highlighted source code of the documented objects.

Because autodoc uses runtime imports while AutoAPI parses files statically, a combined site can show two different API names for the same object (file-of-definition vs. public exposure). This is the central traceability tension the demo highlights.

Typical inputs users provide for each Sphinx path

1. autodoc workflow (runtime/exposure)

- Inputs the user must provide:
 - An importable Python package (it must be on `sys.path` at build time).
 - **docs/conf.py** including extensions = ['sphinx.ext.autodoc', 'sphinx.ext.napoleon', 'autoapi.extension', ...].
 - **.rst** pages with `.. automodule:: mypkg / .. autoclass:: mypkg.Class` directives or **sphinx-apidoc**-generated stubs.
- Build step: `sphinx-build -b html docs docs/_build` (autodoc will import modules to create docs).
- Key config knobs:
 - **autodoc_default_options** (e.g., `members`, `imported-members`, `noindex`).
 - **add_module_names**: how names are prefixed.

2. sphinx-apidoc (automated stub generation)

- Inputs: path to package (must be a Python package), optional excludes.
- Command: `sphinx-apidoc -o docs/ path/to/package`
- Use-case: bootstrapping many automodule `.rst` files before running `sphinx-build`.

3. AutoAPI workflow (static parsing / file-of-definition)

- **Inputs**:
 - `autoapi_type = "python"` and `autoapi_dirs = ['./mypkg']` in `docs/conf.py`.
 - Optionally `autoapi_add_toctree_entry`, `autoapi_keep_files`, `autoapi_ignore` / `autoapi_ignore_patterns`.
- **Behavior**: AutoAPI discovers Python modules by scanning the source files, parsing them, generating `.rst` pages under an `autoapi_root` (e.g., `implementation/`), and creates file-of-definition pages without importing the code.

4. Sphinx-viewcode extension (bidirectional linking)

- Input user has to provide: `extensions = [sphinx.ext.viewcode]` in `docs/conf.py`

Why these inputs matter for traceability:

For traceability (definition → public API path), neither approach alone suffices for large codebases using deep re-exports, wildcard imports, or dynamic `__all__` changes.

- ***sphinx.ext.autodoc*** maps documentation to where the object is imported into the runtime namespace; thus, re-exports and import-time aliasing strongly affect API names users see.
- ***autoapi.extension*** maps documentation to where the object is defined on disk (file-of-definition).

Sphinx's ***viewcode*** extension identifies Python object descriptions (e.g., `.. class::`, `.. function::`) within the reStructured or MyST Markdown documentation, attempts to locate the corresponding source files containing these objects, then, when found, it generates a separate HTML page for each module, featuring a highlighted version of its source code. Finally, it adds a bidirectional link to the highlighted source code and its documentation.

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Demo Project:

We present a minimal package here to gather user perception of the trace-link recovery capabilities of Sphinx and MapCoDoc. We also aim to assess the constraints incurred in coding and documentation practices as a result of the tools' perceived ease of use.

On this page, you will see the repository layout of the project and the implementation of each of its Python modules. Files under ***demo_pkg/docs/**** are responsible for Sphinx operation, which will be shown next.

Note: You are not expected to study the implementation details, but to understand the import-export patterns of the package structure.

Repository Layout:

```
demo_pkg/
├─ demo_pkg/
│   ├── __init__.py
│   ├── helpers.py
│   └─ subpkg/
│       ├── __init__.py
│       └─ layers.py
└─ docs/
    ├── conf.py    (mixed: autodoc + autoapi)
    ├── index.rst
    └─ public_api/
        ├── index.rst
        ├── demo_pkg.rst
        ├── helpers.rst
        └─ subpkg.rst

# AutoAPI will generate an implementation/ subtree during build
```

demo_pkg/__init__.py:

```
import pkgutil, importlib, sys

# Expose only the top-level helper
from .helpers import top_exposed_helper

# Make subpackages discoverable (register in sys.modules) but avoid re-importing:
for finder, name, ispkg in pkgutil.iter_modules(__path__, prefix=__name__ + "."):
    if name not in sys.modules:
        importlib.import_module(name)

__all__ = ["top_exposed_helper", "subpkg"]
```

demo_pkg/helpers.py:

```
def utility_function(x, y):
    """Add two numbers and return the result.

    Desired documented API name: ``demo_pkg.helpers.utility_function``
    """
    return x + y

def top_exposed_helper(z: float, scale: float = 1.0) -> float:
    """
    Top-level helper that scales an input value after multiplying by 10.

    Parameters
    -----
    z : float
        Numeric input value to be processed.
    scale : float, optional
        Scaling factor applied after the base multiplication. Default is
        ``1.0`` which preserves the base result (i.e., ``z * 10``).

    Returns
    -----
    float
        The result of ``(z * 10) * scale``. Always returned as a float.

    Examples
    -----
    Basic usage (top-level import):

    >>> from demo_pkg import top_exposed_helper
    >>> top_exposed_helper(2)
    20.0

    Apply a scaling factor:

    >>> top_exposed_helper(2, scale=0.5)
    10.0

    Notes
    -----
    - The function is intentionally simple.
    - If you need integer-only semantics, cast results explicitly:
      ``int(top_exposed_helper(2, scale=1.0))``.
    """
    return float(z) * 10.0 * float(scale)

__all__ = ["utility_function", "top_exposed_helper"]
```

demo_pkg/subpkg/__init__.py:

```
from .layers import Layer          # re-exported at subpackage level
from . import layers               # keep module object accessible (demo_pkg.subpkg.layers)

__all__ = ["Layer", "layers"]
```


demo_pkg/subpkg/layers.py:

```
class Layer:
    """
    A simple example class representing a layer.

    Desired documented API name (public page): ``demo_pkg.subpkg.Layer``
    (we achieve this by re-exporting Layer from the subpackage __init__)
    """
    def __init__(self, name: str):
        self.name = name

    def forward(self, x):
        """Example forward computation."""
        return f"Processing {x} in layer {self.name}"

def count_layer(seq):
    """
    Count something in a layer.

    Desired documented API name: ``demo_pkg.subpkg.layers.count_layer``
    (kept in layers module so implementation page holds canonical doc)
    """
    return len(seq)

__all__ = ["Layer", "count_layer"]
```

*** 10. Do you have a reasonable understanding of the package structure?**

- ☐ I have a very good understanding
- ☐ I have a fair understanding
- ☐ I am confused

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Sphinx setup for trace-link retrieval:

Here you will see the Sphinx-specific process required for resolving API path and link trace artifacts in the next page.

A brief description of the process:

- Configuration setting inside ``docs/conf.py``:
 - Adding the necessary Sphinx extensions and setting their default options.
 - Adding a wrapper code to exclude specific FQNs of objects so Sphinx doesn't duplicate their tracing and linking.
- Creating ``rst files`` that are not generated by ``autoapi``.
 - Some require stricter implementation to specify that only certain objects can be traced and documented in a module.
- Carry out the build steps.

Note: You are not expected to study the implementation details, but to understand the need for the framework-specific configurations and wrappers illustrated.

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

docs/conf.py:

```
import sys
from pathlib import Path
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

project = "Demo Package (mixed views)"
author = "Survey Example"
release = "0.1"

# -- General configuration ----
extensions = [
    "sphinx.ext.autodoc",      # runtime view (autodoc)
    "sphinx.ext.autosummary",
    "sphinx.ext.napoleon",
    "autoapi.extension",      # static view (AutoAPI)
    "sphinx.ext.viewcode",
]

# Autodoc options (public API view)
autodoc_default_options = {
    "members": True,
    "imported-members": True,
    "undoc-members": False,
    "show-inheritance": True,
    "noindex": True, # prevent autodoc from generating indexed cross-reference targets
}

# -- Ensure autodoc and autoapi coexist gracefully -----
exclude_patterns = ["_build", "Thumbs.db", ".DS_Store", "implementation/_autoapi_templates"]

add_module_names = True # allow API name to start with its exporting/re-exporting module __name__

autoapi_type = "python"
autoapi_dirs = [str(ROOT / "demo_pkg")]
autoapi_keep_files = True
autoapi_root = "implementation"
autoapi_add_toctree_entry = True
autoapi_options = ["members", "show-inheritance"]

templates_path = ["_templates"]
html_theme = "alabaster"
```

- **`autodoc_default_options["imported-members"]`**: shows re-exported names on package pages. (Useful to see Layer under `demo_pkg.subpkg`.)
- **`autodoc_default_options["noindex"] = True`**: avoids duplicate cross-reference target warnings when AutoAPI also creates targets, but it does not remove duplicate pages. To remove duplicates, either additional code to filter the namespace for the duplicated member or applying a stricter `.rst` pattern is required.
- **`autoapi_root = "implementation"`** and **`autoapi_add_toctree_entry = True`**: place AutoAPI-generated implementation pages under `implementation/` so they appear in the same site.
- **`sphinx.ext.viewcode`**: adds a bidirectional link to the highlighted source code of the documented object.

```

# ===== AutoAPI skip handler =====
from sphinx.util.logging import getLogger
logger = getLogger(__name__)

# Fully-qualified names we want to skip in AutoAPI output (exact strings)
AUTOAPI_SKIP_FQNS = {"demo_pkg.subpkg.layers.Layer"}

def autoapi_skip_reexported_member(app, what, name, obj, skip, options):
    try:
        # Build name candidates
        candidates = set()
        # 1) the 'name' argument as received
        if isinstance(name, str):
            candidates.add(name)
        # 2) if obj has __module__ and __name__, add that fullname
        try:
            mod = getattr(obj, "__module__", None)
            nm = getattr(obj, "__name__", None)
            if mod and nm:
                candidates.add(f"{mod}.{nm}")
        except Exception:
            # ignore problems getting attributes from obj
            pass
        # 3) if name is short (no dots) and we determined a mod from obj, add mod.name
        if isinstance(name, str) and "." not in name and 'mod' in locals() and mod:
            candidates.add(f"{mod}.{name}")
        # Now check candidates against explicit skip list
        for c in candidates:
            if c in AUTOAPI_SKIP_FQNS:
                logger.info(f"[autoapi-skip] skipping exact match: {c}")
                return True
        # Heuristic match: skip Layer defined under demo_pkg.subpkg.layers
        # if any candidate indicates that path and the short name is 'Layer'
        if any("demo_pkg.subpkg.layers" in c for c in candidates) and any(c.endswith(".Layer") or c == "Layer" for c in candidates):
            logger.info(f"[autoapi-skip] heuristic skip for Layer; candidates={candidates}")
            return True
    except Exception as exc:
        # Log a single warning but do NOT raise - be conservative in errors
        logger.warning(f"[autoapi-skip] handler error (ignored): {exc!r}")
    # None => let AutoAPI proceed normally
    return None

def setup(app):
    app.connect("autoapi-skip-member", autoapi_skip_reexported_member)
    logger.info(f"[conf.py] registered robust autoapi-skip-member handler")
    return {"version": "0.1", "parallel_read_safe": True}

```

This snippet added to ``docs/conf.py`` registers an `autoapi-skip-member` handler that prevents AutoAPI from generating implementation-level docs for specified members. It works in two ways: (a) an explicit set `AUTOAPI_SKIP_FQNS` of exact fully-qualified names you want to suppress, and (b) a conservative heuristic that detects likely re-exported symbols (for example, `Layer` defined in `demo_pkg.subpkg.layers` but re-exported as `demo_pkg.subpkg.Layer`).

docs/index.rst:

```

Project Documentation (mixed views)
=====

```

The docs site shows both the public API (runtime view) and the implementation (static) view.

```

.. toctree::
   :maxdepth: 2
   :caption: Contents:

   public_api/index
   implementation/index

```

This is the top-level site. ``implementation/index`` will be populated by AutoAPI-generated pages (or you can create manual stubs).

docs/public_api/index.rst:

Public API Reference

=====

This section documents the intended public API as seen by users.

```
.. toctree::
    :maxdepth: 1

    demo_pkg
    helpers
    subpkg
```

- ``automodule`` produces pages that reflect how the package exposes names at import-time.
- ``imported-members``: shows re-exported names (so `Layer` appears under `demo_pkg.subpkg`).
- ``noindex``: is used so autodoc won't create indexed cross-ref targets (avoids duplicate-target warnings when AutoAPI also creates targets), but `:noindex:` does not remove duplicate pages.

docs/public_api/demo_pkg.rst (optional):

demo_pkg Package

=====

```
.. automodule:: demo_pkg
    :members:
    :imported-members:
    :undoc-members:
    :noindex:
```

See also the `:doc:`../implementation/index`` for underlying implementation.

docs/public_api/helpers.rst:

demo_pkg.helpers Module

=====

```
.. autofunction:: demo_pkg.helpers.utility_function
    :noindex:
```

A stricter approach to avoid duplication by excluding ``top_exposed_helper``.

docs/public_api/subpkg.rst:

demo_pkg.subpkg Package

=====

```
.. automodule:: demo_pkg.subpkg
    :members:
    :imported-members:
    :undoc-members:
    :noindex:
```

By setting up the necessary configurations, .rst files, and autoapi-skip-member handler hook, Sphinx **correctly traces** the desired candidate API names and **accurately links them** to their respective docstrings.

Desired canonical API name	Actual Sphinx outputs
demo_pkg.helpers.utility_function	demo_pkg.helpers.utility_function
demo_pkg.top_exposed_helper	demo_pkg.top_exposed_helper
demo_pkg.subpkg.Layer	demo_pkg.subpkg.Layer
demo_pkg.subpkg.layers.count_layer	demo_pkg.subpkg.layers.count_layer

*** 11. How well do you understand the Sphinx configuration process and the RST files that facilitate API name tracing and linking the code body to its docstring?**

- ☐ I have a very good understanding
- ☐ I have a fair understanding
- ☐ I am confused

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Please answer the questions below about the Sphinx demo you just examined (conf.py, .rst files, and expected vs actual API outputs leading to link retrieval). For each statement, select the response that best reflects your judgment for the example you were shown.

*** 12. Sphinx is easy to learn to use in your Python projects.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 13. Sphinx's learning curve negatively impacts software engineers' motivation to create and maintain documentation.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 14. The framework-specific Sphinx configuration (managing extensions, creating .rst files, and adding wrappers to skip objects) is an error-prone process.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 15. The framework-specific Sphinx configurations are easy to maintain in a rapidly evolving software development lifecycle.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly Agree

*** 16. To what extent would you be inclined to change your implementation design to adapt to Sphinx's behavior?**

- ☐ I will produce the necessary configurations to produce the expected result
- ☐ I will slightly modify my implementation design to fit the needs of Sphinx
- ☐ I would make significant changes to my package structure so I can apply the bare minimum Sphinx configurations

*** 17. Do you agree that documentation practices would improve if Sphinx supported trace-link retrieval from source code to external documentation (web and/or file documentation) during initial build?**

- ☐ Yes
- ☐ No
- ☐ Not sure

*** 18. Select all that apply regarding the side effects docstring-dependent trace-link retrieval has on documentation practices.**

- ☐ Cluttered code files due to the practice of writing API reference documentation as docstrings.
- ☐ Potentially introduces code and documentation smells as developers and documentation maintainers make changes to the same artifact.
- ☐ Reduced discoverability and navigability across code and documentation
- ☐ None of the above
- ☐ Other (please specify)

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

MapCoDoc Trace-Link Recovery Pipeline:

MapCoDoc is a configurable hybrid static and sandboxed dynamic analysis pipeline that computes export chains and maps code entities to candidate public API names, then links them to API reference documentation (web-based or PDF). It produces a relational mapping showing where an object is defined and how it is exposed, processes documentation (web-doc or file-doc), and computes a hybrid keyword-semantic search to recover trace-link and produces a relational database of the linked artifacts. It currently only supports Python packages.

*** 19. Would you like to see a brief description of MapCoDoc's input requirement from its user, and the code and documentation analysis workflow it employs to link artifacts?**

☐ Yes

☐ No

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Mapping Code onto Documentation (MapCoDoc) Trace-Link Recovery Pipeline

What MapCoDoc expects as input:

- A Python code repository URL or local path to codebase.
- A URL to published API reference HTML pages for a module/member, or a PDF documentation to link to.
- Configuration: whether to run in static-only, hybrid (static + sandboxed dynamic) code analysis.
- Large language model API key (if working with external documentation: web-doc & file-doc).
- [Optional] Proxies from your proxy provider if you want scraping processes to run in parallel.

Key features:

- Deterministic export chain resolution using dependency-aware wildcard expansion (topological sort).
- Hybrid analysis: static-first, dynamic verification sandbox for cases where `__all__` or import-time logic changes exports.
- Explicit API candidate listing for each code entity (so you can choose programmatically which API name to canonize).
- A hybrid keyword-semantic search-based link retrieval.
- Programmatically structuring and linking documentation of heterogeneous formats to respective code components.

Core pipeline:

1. **File discovery & fully-qualified names (FQN) mapping:** walk repo, map files ↔ module FQN.
2. **Per-file static (Abstract Syntax Tree) analysis:**
 - Collect definitions, imports (including relative/wildcard), `__all__`, and create `ExportRecord` and `ImportRecord`.
 - Other relevant trace units, such as the code body, signature, and docstring, are also extracted.
 - The static analysis is also responsible for scanning for runtime imports and modifications on `__all__` values to trigger the dynamic analysis flag for the module under analysis.
3. **On-the-fly resolution:**
 - While parsing, resolve links when possible; expand wildcards if sources already parsed.
4. **Dynamic analysis:**
 - If the dynamic analysis flag is triggered, the module is run in a sandboxed virtual environment to assess its namespace and identify the final components it exports.
 - An internet connection is required to install project dependencies.

5. ***Dependency-aware final resolution:***

- Build a dependency graph between modules that have wildcard re-exports and run a topological sort to expand wildcard sources in the correct order.

6. ***Chain candidate identification:***

- Identify elements re-exported from other modules and assemble export chains.

7. ***API path resolution:***

- Guided backward trace to find all exposed API names for a component at the top-level module it's exported/re-exported from, returning a set of candidate API names.

8. ***Documentation linking:***

- If provided with API docs (HTML / PDF), MapCoDoc scrapes pages, finds the relevant API page and fragments, and links them back to the code entity.
- For large pages with many members, MapCoDoc uses keyword and/or semantic (as fallback) API name-signature searches, then feeds the retrieved text to a large language model to extract relevant structured reference documentation.

9. ***Output:***

- Relational database with records describing import/export relationships, the API names found, structured documentation, signature variants, code body, and metadata.

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

MapCoDoc setup for trace-link retrieval:

Required inputs from the user:

- URL or local path to:
 - Demo project's repository
 - URL to one of the objects' documentation or PDF (if external documentation exists)
- Enabling hybrid static-dynamic analysis (or leave it as static)
- API key of an LLM (GPT-4o recommended. As of now, a locally run language model is not embedded in the MapCoDoc workflow).
- [Optional] Proxies from your proxy provider if you want scraping processes to run in parallel.

After providing the inputs, the workflow is executed, and a relational database of linked artifacts (source code and documentation) is produced. Here, the trace units for one of the objects (**top_exposed_helper**) in the demo project are shown to illustrate how each module member is represented in the database.

```

{
  "component_kind": "function",
  "access_modifier": "public",
  "component_name": "top_exposed_helper",
  "definition_fqn": "demo_pkg.helpers.top_exposed_helper",
  "definition_module": "demo_pkg.helpers",

  "API_candidates": [
    "demo_pkg.top_exposed_helper",
    "demo_pkg.helpers.top_exposed_helper"
  ],

  "chosen_API_name": "demo_pkg.top_exposed_helper",

  "export_chain": [
    {"module": "demo_pkg.helpers", "exported_as": "top_exposed_helper"},
    {"module": "demo_pkg", "reexported_as": "top_exposed_helper"}
  ],

  "signature": {
    "full": "top_exposed_helper(z: float, scale: float = 1.0) -> float",
    "defaults_only": "top_exposed_helper(z, scale = 1.0)"
  },
  "parameters": [
    {
      "name": "z",
      "type": "float",
      "default": null,
      "is_positional_only": false,
      "is_keyword_only": false,
      "is_vararg": false,
      "is_kwarg": false
    },
    {
      "name": "scale",
      "type": "float",
      "default": "1.0",
      "is_positional_only": false,
      "is_keyword_only": false,
      "is_vararg": false,
      "is_kwarg": false
    }
  ],
  "returns": "float",

  "source": {
    "file_path": "demo_pkg/helpers.py",
    "start_line": 11,
    "end_line": 55
  },
}

```

```

"code_body": "[NOT SHOWN FOR BREVITY]",
"docstring": "[NOT SHOWN FOR BREVITY]",
"API_reference_documentation": {
  "module_member_signature": "top_exposed_helper(z: float, scale: float = 1.0) -> float [source] (URL placeholder to source code)",
  "module_member_description": "Top-level helper that scales an input value after multiplying by 10.",
  "parameters": [
    {
      "name": "z",
      "type": "float",
      "description": "Numeric input value to be processed.",
      "additional_information": "N/A"
    },
    {
      "name": "scale",
      "type": "float (optional)",
      "description": "Scaling factor applied after the base multiplication.",
      "additional_information": "Default is `1.0` which preserves the base result (i.e., `z * 10`)."
    }
  ],
  "returns": {
    "type": "float",
    "description": "The result of `(z * 10) * scale`. Always returned as a float."
  },
  "examples": [
    {
      "example": "`>>> from demo_pkg import top_exposed_helper\n>>> top_exposed_helper(2)\n20.0`",
      "additional_information": "Basic usage (top-level import):"
    },
    {
      "example": "`>>> top_exposed_helper(2, scale=0.5)\n10.0`",
      "additional_information": "Apply a scaling factor:"
    }
  ],
  "additional_notes": {
    "supplementary_information": [
      "The function is intentionally simple.",
      "If you need integer-only semantics, cast results explicitly: `int(top_exposed_helper(2, scale=1.0))`"
    ],
    "edge_cases": []
  }
}

```

Each module member is subjected to a similar analysis as the `top_exposed_helper` function, and is linked to its respective docstring and external documentation.

If a module member is exposed with more than one fully qualified name from the same top-level module in the package, all the candidate API names are traced and extracted along with the definition FQN.

While the shortest candidate FQN alongside the member's signature is first used for trace-link retrieval, the remaining are used if the ones before them fail to establish a link with the documentation.

API names in the external documentation	API name traced by MapCoDoc
demo_pkg.helpers.utility_function	demo_pkg.helpers.utility_function
demo_pkg.top_exposed_helper	Chosen FQN: demo_pkg.top_exposed_helper Candidate FQNs: [`demo_pkg.top_exposed_helper`, `demo_pkg.helpers.top_exposed_helper`]
demo_pkg.subpkg.Layer	Chosen FQN: demo_pkg.subpkg.Layer Candidate FQNs: [`demo_pkg.subpkg.Layer`, `demo_pkg.subpkg.layers.Layer`]
demo_pkg.subpkg.layers.count_layer	demo_pkg.subpkg.layers.count_layer

*** 20. How well do you understand MapCoDoc's traceability workflow and its analysis result?**

- ☐ I have a very good understanding
- ☐ I have a fair understanding
- ☐ I am confused

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Please answer the questions below about the MapCoDoc demo you just examined. For each statement, select the response that best reflects your judgment for the example and workflow you were shown.

*** 21. MapCoDoc is easy to learn to use for your Python project.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 22. Managing MapCoDoc input configuration to produce reliable trace-links is an error-prone process.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly Agree

*** 23. The hybrid keyword-semantic link retrieval makes MapCoDoc adaptable to potential misalignment of signatures and API names in the documentation.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 24. To what extent would you be inclined to change your implementation design to make MapCoDoc produce the desired canonical API names?**

- ☐ I expect little or no source rework.
- ☐ I will make minor code adjustments (small aliases or wrappers) to improve the clarity of export chains.
- ☐ I would make significant structural changes to my package to force a particular canonical API shape.
- ☐ Other (please specify)

*** 25. MapCoDoc supports linking external documentation sources (published HTML pages or PDF documents) to code artifacts; would this capability improve documentation practices in your team?**

- ☐ Yes
- ☐ No
- ☐ Not sure

*** 26. Select all that apply:**

Which of the following are advantages of having a traceability pipeline that supports heterogeneous documentation formats?

- ☐ Reduction of bulky content directly in source code (less docstring bloat).
- ☐ Reducing documentation/code conflicts where developers and documentation maintainers work on different artifacts.
- ☐ Improved discoverability and navigation across code and documentations (single place to find both definitions and API pages).
- ☐ None of the above.
- ☐ Other (please specify)

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

MapCoDoc as a Platform for Tooling Downstream Program Comprehension Tasks:

You will now evaluate how useful MapCoDoc's outputs (export chains, candidate API names, chosen canonical name, structured doc fragments, and relational records) would be for extending tools for various downstream program-comprehension tasks. For each task, indicate how valuable MapCoDoc would be for your team and how likely you would adopt it for that use.

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Code Maintenance and Change-Impact Analysis:

*** 27. MapCoDoc would significantly speed up API evolution analysis and deprecation planning.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 28. MapCoDoc's export-chain records would meaningfully reduce the time it takes to identify which public APIs and aliases are affected by a code refactor.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 29. MapCoDoc would speed up mapping a bug reported against a public API to the exact implementation and other aliases that must be fixed.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 30. MapCoDoc's import and export tracing of internal and external code components in the code repository is useful for identifying unused components and facilitating the adoption of explicit export (via `_all_`).**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Documentation Maintenance:

*** 31. Recovering trace links from heterogeneous document formats makes MapCoDoc valuable for consolidating documentation and ensuring consistency.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 32. MapCoDoc-generated programmatically parseable documentation enables its automated fine-grained quality assessment.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 33. MapCoDoc's fine-grained representation of documentation facilitates automated tooling for targeted documentation maintenance/generation.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Developer Onboarding and Comprehension:

*** 34. Powering IDE hover/popups with MapCoDoc's fine-grained representation of artifacts and provenance information reduces the required effort to learn about a codebase.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 35. An AI assistant (agent) that answers developers' questions by querying MapCoDoc's analysis results would generate more accurate responses and consume fewer resources than one searching a raw code repository and documentation.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Cross-Language Code Translation:

*** 36. Relational representation of code, documentation, and import dependencies is critical for component architecture-informed cross-language translation (around public-facing APIs).**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 37. Compared to the manual discovery-translation workflow, using MapCoDoc's output to produce translation specifications and scaffolding would reduce the total time required to port a module/package.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 38. MapCoDoc's output would be a high-quality input to an automated translator or LLM to generate a translation specification.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

Machine Learning Dataset and Research Utilities:

*** 39. MapCoDoc's relational representation of trace artifacts serves as a foundation for building a benchmark dataset for code-to-documentation.**

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

*** 40. Select all that apply: Which use-cases do you consider relevant about MapCoDoc's output as a dataset?**

- ☐ To train or fine-tune models to generate high quality API reference documentations.
- ☐ To detect stale or mismatched documentations.
- ☐ To train models to generate accurate API usage examples.
- ☐ To train models to accurately retrieve trace-links between code and documentation.
- ☐ To perform provenance / governance research (identify third-party re-exports and licensing exposure)
- ☐ To estimate maintenance effort by analyzing imports dependencies and export records.
- ☐ None of the above.
- ☐ Other (please specify)

MapCoDoc: Fine-Grained Trace Link Recovery Between Source Code and API Reference Documentation

End of survey.

Thank you for your valuable time and feedback!